DELTA CODE 2026 | ROUND 2 | PROBABILITY & STATISTICS

k-Way Collision Probabilities

High-dimensional vectorized Monte Carlo vs. Poisson analysis

Team Enthusiast
Krish Agrawal
krishagrawal222@gmail.com

Technozion 2026 | The Mathematical Society, NIT Warangal

The problem

Generalized birthday: when does some bucket get k or more keys?

$$P(\text{Collision} \geq k) \approx 1 - \exp(-C(n,k) / C^{(k-1)})$$

n keys drawn uniformly from C buckets | k = 2 is the classic birthday problem | $\lambda = C(n,k) / C^{(k-1)}$

Why it matters

Hash collisions (SHA-256), salts and database sharding rely on collision resistance over huge key spaces, up to $C = 2^{32}$.

The hard part

For $k \geq 3$ the events are rare ($p < 10^{-6}$). Brute-force Monte Carlo breaks down and analysis must take over.

Our approach

Derive the Poisson bound, simulate with vectorized NumPy, then quantify exactly where each method works.

Why Poisson works

Derivation: count colliding k-subsets

1. Exact reformulation

Let X be the number of k-subsets of keys that all share a bucket. Then {max load ≥ k} = {X ≥ 1} exactly.

2. Mean of X

Each subset collides with probability C^−(k−1), so E[X] = C(n,k) / C^(k−1) = λ.

3. Poisson limit (Chen-Stein)

Only overlapping subsets are dependent. The error is bounded by $\min(1, 1/\lambda)\cdot(b_1+b_2)$, with $b_2 \approx \lambda\cdot k\cdot n/C$, so it vanishes when $n \ll C$.

4. Where it is biased

k = 2: Poisson slightly under-estimates (≈ n/3C). k ≥ 3: it over-counts buckets with load > k, so it over-estimates as λ grows.

Vectorized Monte Carlo engine

No Python loop over trials; memory independent of T

Detect

Sort each row in place, then test S[:, k−1:] == S[:, :n−k+1]. In a sorted row, a value occurs ≥ k times iff keys i and i+k−1 match. np.diff and bincount versions cross-check it.

Chunk

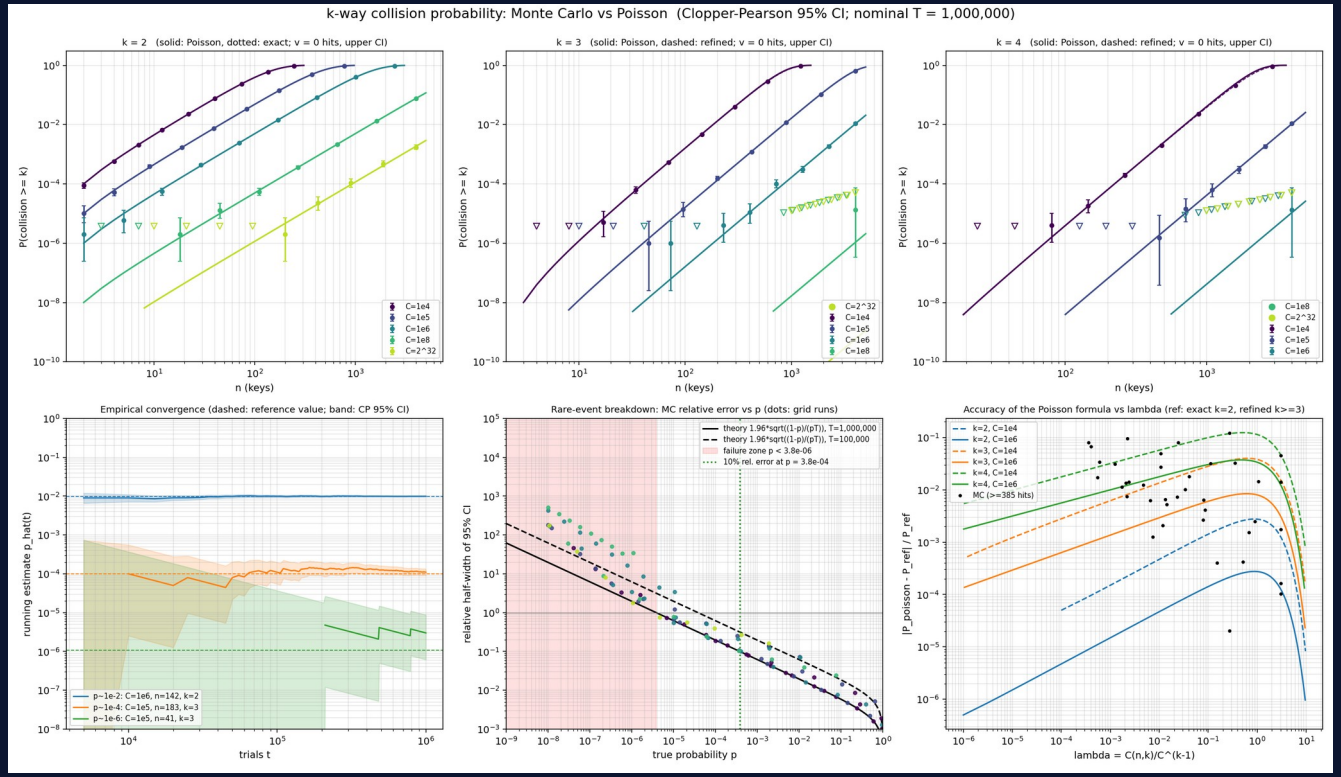
Full T×n matrix is 4 GB at n = 1000. Block size comes from a memory budget (256 MB default), keys are uint32 for $C \le 2^{32}$, rows are C-contiguous, and results are bit-identical for any chunk size.

Verify

15 self-tests pass, including exact enumeration. Clopper-Pearson and Wilson 95% CIs via scipy.stats. NumPy beat a numba kernel about 7× on 1 core, so NumPy is the default.

Results

135 parameter points: C from 10^4 to 2^{32}, k = 2, 3, 4



135

grid points with 95% CP and Wilson CIs

$2.8 \cdot 10^{10}$

random keys simulated in 277 s

15 / 15

self-tests pass

Where Monte Carlo breaks down

Relative error ≈ 1.96 / √(p·T); with T = 10^6:

$3.8 \cdot 10^{-4}$

below this p, estimates are worse than ±10%

$3.8 \cdot 10^{-6}$

CI as wide as the estimate itself (failure zone)

60 / 135

grid points returned exactly zero hits

Empirical confirmation

39 points reliable (≥ 385 hits), 22 noisy, 74 failed (< 10 hits). At p ≈ 10^{-6} the running estimate used 3 hits and was off by 2.8×.

Hand-over to analysis

For C up to 2^{32} and k ≥ 3, every feasible n has p below the failure zone. Only the Poisson formula can answer there, with relative error ≈ 10^{-6}.

Findings and takeaways

Poisson accuracy

Excellent for $k = 2$ at large C. For $k \geq 3$ it over-estimates: 0.950 vs 0.908 at $\lambda = 3$, $k = 4$, $C = 10^4$ (MC CI excludes Poisson).

Complementary regimes

Monte Carlo covers $p \gtrsim 10^{-4}$; Poisson covers $p \lesssim 10^{-5}$. A refined bucket-occupancy formula matched the MC at 129 / 135 points.

Deliverables

birthday.py, results.csv, plot_grid.png, report.md, requirements.txt, README.md, public GitHub repo.

Team Enthusiast

Krish Agrawal
krishagrawal222@gmail.com
Thank you!